Higher Order Functions

CMSC 323: Design and Implementation of Programming Languages

This lecture note is adapted from content created by Robby Findler (Northwestern University) and Vincent St-Amour (Northwestern University) and Shriram Krishnamurthi in Programming Languages: Application and Interpretation.

Higher Order Functions

Functions that accept other functions as arguments and return functions as values

Why Higher Order Functions?

- Recall what is meant by “higher order function”
  - Functions can be assigned to a variable
  - Passed as a parameter to another function
  - Returned from a function
  - Stored inside data structures (like arrays or maps)

Why Higher Order Functions?

- Abstraction is easier with functions as values
- Say what?
  - First-order functions only allow you to pass data values into a reusable block of code. Higher-order functions allow you to pass behavior.

Example

javascript

```javascript
// First-order helpers (abstracts data transformation)
function isActive(user) {
    return user.status == "active";
}

function getEmail(user) {
    return user.email;
}

// The execution logic: Hides nothing about HOW to iterate
function getActiveUserEmails(users) {
    let results = [];
    for (let i = 0; i < users.length; i++) {
        if (isActive(users[i])) { // Hardcoded control logic (conditional
            results.push(getEmail(users[i])); // Hardcoded state mutation
        }
    }
    return results;
}
```

If you later want to filter active users to get their phone number instead, you have to copy-past the entire loop structure

Example

javascript

```javascript
// Generic HOF that accepts the behavioral action as an input parameter
function getActiveUserProperty(users, extractBehavior) {
    let results = [];
    for (let i = 0; i < users.length; i++) {
        // Using our helper to check the condition
        if (isActive(users[i])) {
            // The machinery simply executes the behavior passed to it
            results.push(extractBehavior(users[i]));
        }
    }
    return results;
}

// The clean, independent behavioral inputs
const isActive = (user) => user.status === "active";
const getEmail = (user) => user.email;
const getPhone = (user) => user.phone;

// Usage:
getActiveUserProperty(users, getEmail);
getActiveUserProperty(users, getPhone);
```

If you later want to filter active users to get their phone number instead, pass a different behavior to the method.

Why Higher Order Functions?

- Abstraction is easier with functions as values
- You've already done some of this: E.g., map, foldl
- Abstract over plus and sub in interp

```
(define (apply op lhs rhs env fundefs)
  (op (interp lhs env fundefs) (interp rhs env fundefs)))
```

op could be plus or sub (or something else)

Why Higher Order Functions?

- Callbacks are possible with higher order functions
- A ***callback*** is a function passed to another function that may be run after the run of another function is completed.

```
playBackInterval = setInterval(playBack, 1000);
```

Doesn't have to be at the end, but typically is run at the end of the function it is passed to, often to signal the end of the execution of the first function.

Why Higher Order Functions?

A separate deffun form becomes unnecessary. We can bind unnamed functions to variable using with.

More generally, if a function is only used once, or a limited number of times, an anonymous function (e.g., lambda) may be syntactically lighter than using a named function. Anonymous functions are ubiquitous in functional programming languages and other languages with first-class functions, where they fulfill the same role for the function type as literals do for other data types.

Why Higher Order Functions?

A separate deffun form becomes unnecessary. We can bind unnamed functions to variable using with.

So what does this mean for us and our language? Up to this point, we have function definitions (defined using deffun) separate from our other syntactic structures (e.g, F1WAE). The text in red says we don't need this separation (or the complications that come with having function definitions separate from other expressions.)

Defining the F1WAE Grammar

<F1WAE> := <num>

| {+ <F1WAE> <F1WAE>}

| {- <F1WAE> <F1WAE>}

|{with {<id><F1WAE>} <F1WAE>}

|<id>

|{<id> <F1WAE>}

|{lam {<id>} <F1WAE>}

Extending our grammar to introduce functions (function definitions)

Why Higher Order Functions?

A separate deffun form becomes unnecessary. We can bind unnamed functions to variable using with.

```
{deffun {f x} {+ 1 x}}
{f 10}
```

```
{with {f {lam {x} {+ 1 x}}}
      {f 10}}
```

Exercise

Change the following deffun form to an 'equivalent' higher order function named with a 'with' form

deffun form

```
{deffun {twice y}
        {+ y y}}

{twice 5}
```

with form

```
{with {twice {lam {y}
                  {+ y y}}}
      {twice 5}}
```

Exercise

Change the following deffun form to an equivalent higher order function named with a 'with' form

```
{deffun {f x}                                    {f 10}
          {- 20 {twice
                   {twice x}}}}

{deffun {twice y}
        {+ y y}}
```

Exercise

Change the following deffun form to an equivalent higher order function named with a 'with' form

```
{deffun {f x}                                    {f 10}
          {- 20 {twice
                   {twice x}}}}

{deffun {twice y}
        {+ y y}}
```

Exercise

Change the following deffun form to an equivalent higher order function named with a 'with' form

```
{deffun {f x}
          {- 20 {twice
                  {twice x}}}}

{deffun {twice y}
        {+ y y}}
```

```
{f 10}

{with {f {lam {x}
              {- 20
                 {with {twice {lam {y}
                                   {+ y y}
                                {{twice x}}}}}}}}
 {f 10}}
```

Defining the F1WAE Grammar

<F1WAE> := <num>

| {+ <F1WAE> <F1WAE>}

| {- <F1WAE> <F1WAE>}

|{with {<id><F1WAE>} <F1WAE>}

|<id>

|{<id> <F1WAE>}

|{lam {<id>} <F1WAE>}

Extending our grammar to introduce functions (function definitions)

F1WAE Evaluation

10 => 10

{+ 1 2} => 3

{- 1 2} => -1

{with {x 7} {+ x 2}} => {+ x 2} => 9

y => free identifier

{lam {x} {+ 1 x}} => {lam {x} {+ 1 x}}

;interp : F1WAE? ... -> F1WAE-Value

What has been the type returned by our interpreter?

What would a lam return?

Our result is not always a number!

We need function values too!

Functions as Values

Python Example: What do these two print statements return?

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)

print(f)
print (f(5))
```

Functions as Values

Python Example: What do these two print statements return?

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)

print(f)
print (f(5))
```

```
>>> def f(n):
...     if n==0:
...         return 1
...     return n * f(n-1)
...
>>> print(f)
<function f at 0x105915dd0>
>>> print(f(5))
120
>>>
```

Functions as Values in Python

We can also bind variables to functions in Python

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

```
g = f
print(g(2))
```

Functions as Values in Python

We can define anonymous functions in Python

```
k = lambda n: n + n


print(k) # prints the function location
print(k(7)) # invokes k and prints 14
```

F1WAE Evaluation

We should be able to invoke a function either with its name or its definition

Function name invocation

```
{with {f {lam {x} {+ 1 x}}}
      {f 3}}
=> {f 3}
```

Function definition invocation

```
{{lam {x} {+ 1 x}} 3}
```

The two expressions should produce the same result. (that is, both should be invoking `{f 3}`, resulting in 4

We update our grammar so that this is possible.

F1WAE Evaluation

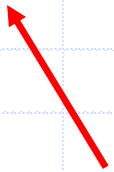
We should be able to invoke a function either with its name or its definition

Function name invocation

```
{with {f {lam {x} {+ 1 x}}}
      {f 3}}
=> {f 3}
```

Function definition invocation

```
{{lam {x} {+ 1 x}} 3}
```



In JavaScript, a function invocation with its definition is known as an Immediately Invoked Function Expression (IIFE)

Defining the F1WAE Grammar

```
<F1WAE> := <num>
         | {+ <F1WAE> <F1WAE>}
         | {- <F1WAE> <F1WAE>}
         |{with {<id><F1WAE>} <F1WAE>}
         |<id>
         |{<id> <F1WAE>}
         |{lam {<id>} <F1WAE>}
         |{<F1WAE> <F1WAE>}}
```

- We include grammar for our function application.
- Function introductions go with function use (or elimination or application).
- We use a function value to invoke a function

Note: we no longer have function names.

Function values need to carry their deferred substitution!

(interp {{with {y 10} {lam {x} {+ y x}}}
{with {y 7} y}})

Argument Expression:

y = 7

(interp {with {y 7} y}}) => (interp y) => 7

Function Expression:

(interp {{with {y 10} {lam {x} {+ y x}}})

y = 10

(interp {lam {x} {+ y x}}) => ?

Now functions can be created in local scopes. They need to remember the environment in which they were defined.

{lam {x} {+ y x}} should remember that y has a value in the scope in which the function was defined.

F1WAE Values

Functions remember the environment in which they were defined

A function along with its environment (deferred subst.) is known as a Closure.

```
(define-type F1WAE-Value
  [numV (n : Number)]
  [funV (param : Symbol)
        (body : F1WAE)
        (env : (Hashof Symbol F1WAE-Value))])
```

Thus, our function value is a closure.

```
(test (interp (parse `{with {y 10} {lam {x} {+ y x}}}) (hash empty))
(funV 'x (plus (var 'y) (var 'x)) (hash (list (pair 'y (numV 10))))))
```

Note the similarity with what Python returned when asked to interpret a function

Function values need to carry their deferred substitution!

```
(interp {{with {y 10} {lam {x} {+ y x}}}
        {with {y 7} y}})
```

Function : {lam {x} {+ y x}}

y = 10

Argument: 7

(interp {+ y x})

x = 7, y = 10

The body's hash map (implementation of deferred substitution) includes the closures hash map content and the parameter-argument pair.

Just like in F1WAE, do not use the deferred substitution from the call environment!

Function Values

What is the result of evaluating this expression?

```
{with {x 1}
     {with {f {lam {y} x}}
          {with {x 2}
               {f 10}}}}
```

Function Values

What is the result of evaluating this expression

```
{{with {x 3} {lam {y} {+ x y}}} 4}
```

Function Values

What is the result of evaluating this expression

```
{{with {y 3} {lam {y} {+ y 1}}} 5}
```

F1WAE Datatype

```
(define-type F1WAE
  [num (n : Number)]
  [sub (lhs : F1WAE) (rhs : F1WAE)]
  [plus (lhs : F1WAE) (rhs : F1WAE)]
  [var (id : Symbol)]
  [with (id : Symbol) (named-expr : F1WAE) (body : F1WAE)]
  [lam (param : Symbol) (body : F1WAE)]
  [app (func : F1WAE)(arg : F1WAE)])

(test (parse `{lam {x} {+ x 1}})
              (lam 'x (plus (var 'x) (num 1))))
```

F1WAE Datatype

```
(define-type F1WAE
  [num (n : Number)]
  [sub (lhs : F1WAE) (rhs : F1WAE)]
  [plus (lhs : F1WAE) (rhs : F1WAE)]
  [var (id : Symbol)]
  [with (id : Symbol) (named-expr : F1WAE) (body : F1WAE)]
  [lam (param : Symbol) (body : F1WAE)]
  [app (func : F1WAE)(arg : F1WAE)])

(test (parse `{{lam {x} {+ x 1}} 10})
              (app (lam 'x (plus (var 'x) (num 1))) (num 10)))
```

F1WAE Datatype

What would be the result of parsing the following to our F1WAE Datatype?

{{with {x 3} {lam {y} {+ x y}}} 4}

F1WAE Datatype

What would be the result of parsing the following to our F1WAE Datatype?

```
{{with {y 3} {lam {y} {+ y 1}}} 5}
```

F1WAE Datatype

What would be the result of parsing the following to our F1WAE Datatype?

```
{with {x 1}
     {with {f {lam {y} x}}
          {with {x 2}
               {f 10}}}}
```

F1WAE Datatype

What is the result of interpreting the F1WAEs in the previous 3 slides above?

Exercise

- Convert the deffuns in the slides below to lams and apps so they still produce the same results as the deffuns
- See the example in the first slide
- Complete the third column in the tables specifying the content of the hash map before interpreting the expressions in first column

Question 1

deffun Form:	Equivalent HOF:
{deffun {f x} x} {with {x 3} {with {y 2} {f x}}}	{with {f {lam {x} x}} {with {x 3} {with {y 2} {f x}}}}}

Expression	description	Hash map while the expression is being interpreted
3 in {x 3}	The named expression in the outer with	f : (x, x, empty)
{with {y 2} {f x}}	The body of the second with	f : (x, x, empty), x : 3
2 in {y 2}	The named expression in the inner with	f : (x, x, empty), x : 3
x in {f x}	The argument in the function application	f : (x, x, empty), x: 3, y : 2
x when executing the body of the lam	The variable in the body of the function f	x : 3

Notation for closures -> <funcName>: (param, body AST, saved environment)

Question 2

deffun Form:	Equivalent HOF:
{deffun {f y} y} {with {x 3} {with {y 2} {f x}}}	

Expression	description	Hash map while the expression is being interpreted
3 in {x 3}	The named expression in the outer with	
{with {y 2} {f x}}	The body of the second with	
2 in {y 2}	The named expression in the inner with	
x in {f x}	The argument in the function application	
y in the body of the lam (when executing it)	The variable in the body of f	

Answer:

Question 2

deffun Form:	Equivalent HOF:
{deffun {f y} y} {with {x 3} {with {y 2} {f x}}}	{with {f {lam {y} y}}

Expression	description	Hash map while the expression is being interpreted
3 in {x 3}	The named expression in the outer with	
{with {y 2} {f x}}	The body of the second with	
2 in {y 2}	The named expression in the inner with	
x in {f x}	The argument in the function application	
y in the body of the lam (when executing it)	The variable in the body of f	

Answer:

Question 2

deffun Form:	Equivalent HOF:
{deffun {f y} y} {with {x 3} {with {y 2} {f x}}}	

Expression	description	Hash map while the expression is being interpreted
3 in {x 3}	The named expression in the outer with	f: (y, y, empty)
{with {y 2} {f x}}	The body of the second with	f: (y,y, empty), x:3
2 in {y 2}	The named expression in the inner with	f: (y, y, empty), x:3
x in {f x}	The argument in the function application	f: (y, y, empty), x:3, y:2
y in the body of the lam (when executing it)	The variable in the body of f	y:3

Answer:

The interpreter jumps inside the body of f. It discards the calling environment (x:3, y:2) and loads f's saved environment (empty). It then binds the formal parameter y to the argument value that was passed in, which was 3.

Question 3

deffun Form:	Equivalent HOF:
{deffun {f y} y} {deffun {g x} {+ x x}} {with {x 3} {with {y 2} {g {f x}}}}	

Expression	description	Hash map while the expression is being interpreted
x in {f x}	The argument in the function application	
y in the body of the lam	The variable in the body of the function (f)	
x in {+ x x} when executing the body of the function	The variable in the body of the function (g)	

Answer:

Question 3

deffun Form:	Equivalent HOF:
{deffun {f y} y} {deffun {g x} {+ x x}} {with {x 3} {with {y 2} {g {f x}}}}	{with {f {lam {y} y}} {with {g {lam {x} {+ x x}}} {with {x 3} {with {y 2} {g {f x}}}}}}

Expression	description	Hash map while the expression is being interpreted
x in {f x}	The argument in the function application	f : (y, y, empty), g : (x, {+ x x}, empty), x : 3, y : 2
y in the body of the lam	The variable in the body of the function (f)	f : (y, y, empty), g : (x, {+ x x}, empty), y : 3
x in {+ x x} when executing the body of the function	The variable in the body of the function (g)	f : (y, y, empty), g : (x, {+ x x}, empty), x : 3

Answer: 6

Question 4

deffun Form:	Equivalent HOF:
{deffun {f y} {+ 10 y}} {deffun {g x} {+ x x}} {with {x 3} {with {y 2} {g {f x}}}}	

Expression	description	Hash map while the expression is being interpreted
x in {f x}	The function argument in the function application	
y in {+ 10 y} when executing f	The variable in the body of the function (lam) f	
x in {+ x x} when executing the body of g	Any of the xes in the body of the function (lam) g	

Answer:

Question 4

deffun Form:	Equivalent HOF:
{deffun {f y} {+ 10 y}} {deffun {g x} {+ x x}} {with {x 3} {with {y 2} {g {f x}}}}	{with {f {lam {y} {+ 10 y}}} {with {g {lam {x} {+ x x}}} {with {x 3} {with {y 2} {g {f x}}}}}}

Expression	description	Hash map while the expression is being interpreted
x in {f x}	The function argument in the function application	f: (y, {+ 10 y}, empty), g: (x, {+ x x}, empty), x : 3, y : 2
y in {+ 10 y} when executing f	The variable in the body of the function (lam) f	
x in {+ x x} when executing the body of g	Any of the xes in the body of the function (lam) g	

Answer:

Question 5

deffun Form:	Equivalent HOF:
{deffun {f y} {+ 10 y}} {deffun {g x} {+ x x}} {with {x 3} {with {y 2} {g {f y}}}}	

Expression	description	Hash map while the expression is being interpreted
y in {f y}	The function argument in the function application	
y in {+ 10 y}	The variable in the body of the function (lam) f	
x in {+ x x} when executing the body of g	Any of the xes in the deffun body the function (lam) g	

Answer:

Question 6

deffun Form:	Equivalent HOF:
{deffun {f x} {with {y 11} y}} {with {x 3} {with {y 2} {f x}}}	

Expression	description	Hash map while the expression is being interpreted
x in {f x}	The function argument in the function application	
y in {with {y 11} y} when executing the body of f	The body of the deffun (f)	

Answer:

Question 7

deffun attempt:	HOF version (slide 31):
{deffun {f y} x} {with {x 1} {with {x 2} {f 10}}}	{with {x 1} {with {f {lam {y} x}} {with {x 2} {f 10}}}}

Expression	description	Hash map while the expression is being interpreted
`{lam {y} x}`	The named expression in the middle with (this creates f's closure)	
2 in `{x 2}`	The named expression in the inner with	
`10` in `{f 10}`	The argument in the function application	
x when executing the body of the lam	The variable in the body of f	

Answer (HOF): Answer (deffun):

Question 7

deffun attempt:	HOF version (slide 31):
{deffun {f y} x} {with {x 1} {with {x 2} {f 10}}}	{with {x 1} {with {f {lam {y} x}} {with {x 2} {f 10}}}}

Expression	description	Hash map while the expression is being interpreted
`{lam {y} x}`	The named expression in the middle with (this creates f's closure)	x : 1
2 in `{x 2}`	The named expression in the inner with	x : 1, f : (y, x, {x : 1})
`10` in `{f 10}`	The argument in the function application	x : 2, f : (y, x, {x : 1})
x when executing the body of the lam	The variable in the body of f	x : 1, y : 10

Answer (HOF): 1 Answer (deffun): error, x is unbound in f's body

F1WAE Interpreter

```
(interp : (F1WAE (Hashof Symbol F1WAE-Value) -> F1WAE-Value))
(define (interp f1wae env)
 (type-case F1WAE f1wae
   [(num n) (numV n)]
                . . .
))
```

Interp a num and return a numV

F1WAE Interpreter

```
(interp : (F1WAE (Hashof Symbol F1WAE-Value) -> F1WAE-Value))
(define (interp f1wae env)
 (type-case F1WAE f1wae
  [(num n) (numV n)]
  [(sub lhs rhs) (numV (apply - (interp lhs env) (interp rhs env)))]
  [(plus lhs rhs) (numV (apply + (interp lhs env) (interp rhs env)))]
                . . .))
```

```
(define (apply op lhs rhs)
(op (type-check lhs) (type-check rhs)))


(define (type-check f1waeV)
(type-case F1WAE-Value f1waeV
[(numV n) n]
[else (error 'type-check "not a number")]))
```

- Interp the lhs and rhs of sub and plus and return a F1WAE-Value
- A funV value is possible – do a type check to return numV only

F1WAE Interpreter

```
(interp : (F1WAE (Hashof Symbol F1WAE-Value) -> F1WAE-Value))
(define (interp f1wae env)
 (type-case F1WAE f1wae
                . . .
  [(var id) (lookup id env)]
                . . .
))
```

Since substitution is deferred, evaluation of variables happen in var. The id is looked up in the environment

F1WAE Interpreter

```
(interp : (F1WAE (Hashof Symbol F1WAE-Value) -> F1WAE-Value))
(define (interp f1wae env)
 (type-case F1WAE f1wae
                . . .
  [(with id named-expr body) (interp body (hash-set env id (interp named-expr env)))]
                . . .
))
```

Add id and interpreted named-expr to the environment for interpreting the body

F1WAE Interpreter

```
(interp : (F1WAE (Hashof Symbol F1WAE-Value) -> F1WAE-Value))
(define (interp f1wae env)
 (type-case F1WAE f1wae
                . . .
  [(lam param body) (funV param body env)]
                . . .
))
```

An anonymous function returns a function value that encloses the environment in which the function was created.

F1WAE Interpreter

```
(interp : (F1WAE (Hashof Symbol F1WAE-Value) -> F1WAE-Value))
(define (interp f1wae env)
 (type-case F1WAE f1wae
                . . .
  [(app a-func arg) (local [(define a-funv (function-type-check (interp a-func env)))
                                                                  . . .
                            ] . . .)]))
```

Interp the function. It returns a function value with parameter, body and an environment (a hash-map)

```
(define (function-type-check f1waeV)
 (type-case F1WAE-Value f1waeV
  [(funV param body env) f1waeV]
  [else (error 'function-type-check "not a function")]))
```

Since have two variants of the F1WAE-Value type,

we check for the one

F1WAE Interpreter

```
(interp : (F1WAE (Hashof Symbol F1WAE-Value) -> F1WAE-Value))
(define (interp f1wae env)
 (type-case F1WAE f1wae
            . . .
            [(app a-func arg) (local [(define a-funv (function-type-check (interp a-func env)))
                                      (define arg-val (interp arg env))
                                                              . . .]
                                                   . . .)])]))
```

Interpret the argument

F1WAE Interpreter

```
(interp : (F1WAE (Hashof Symbol F1WAE-Value) -> F1WAE-Value))
(define (interp f1wae env)
 (type-case F1WAE f1wae
                . . .
  [(app a-func arg) (local [(define a-funv (function-type-check (interp a-func env)))
                            (define arg-val (interp arg env))
                            (define fun-body (funV-body a-funv))
                                                    . . .]
              . . .)]))
```

Retrieve the function body from the function value

F1WAE Interpreter

```
(interp : (F1WAE (Hashof Symbol F1WAE-Value) -> F1WAE-Value))
(define (interp f1wae env)
 (type-case F1WAE f1wae
                . . .
  [(app a-func arg) (local [(define a-funv (function-type-check (interp a-func env)))
                            (define arg-val (interp arg env))
                            (define fun-body (funV-body a-funv))
                            (define param (funV-param a-funv))
                            . . .]
                      . . .)]))
```

Retrieve the function's parameter

F1WAE Interpreter

```
(interp : (F1WAE (Hashof Symbol F1WAE-Value) -> F1WAE-Value))
(define (interp f1wae env)
 (type-case F1WAE f1wae
                . . .
  [(app a-func arg) (local [(define a-funv (function-type-check (interp a-func env)))
                            (define arg-val (interp arg env))
                            (define fun-body (funV-body a-funv))
                            (define param (funV-param a-funv))
                            (define nv (funV-env a-funv))]
                      . . .)]))
```

Retrieve the function's enclosing environment

F1WAE Interpreter

```
(interp : (F1WAE (Hashof Symbol F1WAE-Value) -> F1WAE-Value))
(define (interp f1wae env)
 (type-case F1WAE f1wae
                . . .
  [(app a-func arg) (local [(define a-funv (function-type-check (interp a-func env)))
                            (define arg-val (interp arg env))
                            (define fun-body (funV-body a-funv))
                            (define param (funV-param a-funv))
                            (define nv (funV-env a-funv))]
                      (interp fun-body (hash-set nv param arg-val)))]))
```

Interpret the body using the closure environment and the parameter-value pair generated

F1WAE Interpreter

```
(interp : (F1WAE (Hashof Symbol F1WAE-Value) -> F1WAE-Value))
(define (interp f1wae env)
 (type-case F1WAE f1wae
  [(num n) (numV n)]
  [(sub lhs rhs) (numV (apply - (interp lhs env) (interp rhs env)))]
  [(plus lhs rhs) (numV (apply + (interp lhs env) (interp rhs env)))]
  [(var id) (lookup id env)]
  [(with id named-expr body) (interp body (hash-set env id (interp named-expr env)))]
  [(lam param body) (funV param body env)]
  [(app a-func arg) (local [(define a-funv (function-type-check (interp a-func env)))
                            (define arg-val (interp arg env))
                            (define fun-body (funV-body a-funv))
                            (define param (funV-param a-funv))
                            (define nv (funV-env a-funv))]
                      (interp fun-body (hash-set nv param arg-val)))]))
```

‘app’ Can Do What ‘with’ Can

Example 1:

{with {x 10} x}

Is the same as

{{lam {x} x} 10}

Example 2:

{with {x 10} {+ x 1}}

Is the same as

{{lam {x} {+ x 1}} 10}

‘app’ Can Do What ‘with’ Can

In general,

{with {<id> < F1WAE >$_1$} < F1WAE >$_2$}

Is the same as:

{{lam {<id> } < F1WAE >$_2$} < F1WAE >$_1$}

We will get rid of with for now

Like IIFEs (immediately Invoked Function Expressions) in JavaScript

Exercise: app can do what with can

Convert every ‘with’ in the expression below to an app

```
{with {x 1}
     {with {f {lam {y} x}}
          {with {x 2}
               {f 10}}}}
```

FAE Grammar

<FAE> := <num>

| {+ <FAE> <FAE>}

| {- <FAE> <FAE>}

~~| {with {<id> <FAE>} <FAE>}~~

| <id>

| {lam {<id>} <FAE>}

| {<FAE> <FAE>}}

FAE Interpreter

```
(interp : (F1WAE (Hashof Symbol F1WAE-Value) -> F1WAE-Value))
(define (interp f1wae env)
 (type-case F1WAE f1wae
   [(num n) (numV n)]
   [(sub lhs rhs) (numV (apply - (interp lhs env) (interp rhs env)))]
   [(plus lhs rhs) (numV (apply + (interp lhs env) (interp rhs env)))]
   [(var id) (lookup id env)]
   [(lam param body) (funV param body env)]
   [(app a-func arg) (local [(define a-funv (function-type-check (interp a-func env)))
                             (define arg-val (interp arg env))
                             (define fun-body (funV-body a-funv))
                             (define param (funV-param a-funv))
                             (define nv (funV-env a-funv))]
                       (interp fun-body (hash-set nv param arg-val)))]))
```

FAE

Multiple parameter functions

Function Definitions

{lam {x y} {+ x y}} is the same as:

{lam {x} {lam {y} {+ x y}}}

FAE

Multiple parameter functions

Function Applications

{{{lam {x} {lam {y} {+ x y}}} 10} 5}

Exercise

Convert this multiple-parameter expression to nested, single-parameter expression.

```
{lam {x y z} {+ x {+ y z}}}
```

FAE Grammar

<FAE> := <num>

| {+ <FAE> <FAE>}

| {- <FAE> <FAE>}

|{with {<id><FAE>} <FAE>}

|<id>

|{lam {<id>} <FAE>}

|{<FAE> <FAE>}

Can we still write recursive functions with this grammar that uses anonymous functions and no “withs”s?